Redis and Celery Implementation Guide

This guide details the implementation of Redis Caching and Celery Asynchronous Tasks (including the polling mechanics of the CSV export task and Celery Beat scheduled batch jobs) within the Placement Portal Application - V2.

---

1. Redis Caching Architecture

Caching is used to minimize slow database reads for repetitive endpoints. In this project, the Administrator Statistics endpoint is cached in Redis.

1.1. Caching Infrastructure and Fallbacks

The caching utility is defined in [`backend/utils/cache.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/utils/cache.py). The connection is initialized with safety guards:

```
try:
    r = redis.Redis(host="localhost", port=6379, db=0, decode_responses=True)
except Exception:
    r = None
```

If the Redis server is not running, the application catches the connection exception and assigns `r = None`. All cache getter, setter, and clearing helpers return safely under `if not r:`:

- `cache_get(key)`: Retrieves value, loads from JSON, handles errors by returning `None`.
- `cache_set(key, value, timeout)`: Stores key-value in Redis with an expiration limit, converting objects to JSON string.
- `cache_clear_pattern(pattern)`: Scans for matching wildcard keys (e.g. `admin_stats:`) and deletes them.

1.2. Custom API Caching Decorator

The custom decorator `@cached_api(prefix, timeout)` intercepts Flask requests:

1. It constructs a unique cache key incorporating the prefix, the query path, and any query string arguments: `cache_key = f"{key_prefix}:{request.path}:{request.query_string.decode('utf-8')}"`
2. It queries Redis for the key. If found (Cache Hit), it converts the cached string back to JSON and returns it immediately with status `200 OK`.
3. If not found (Cache Miss), it executes the original route function, extracts the result JSON, stores it in Redis with the given expiration timeout, and returns the response.

1.3. Cache Invalidation

When database changes occur, the cached statistics become stale. To solve this, cache invalidation clears the keys using wildcards:

- File: [`backend/routes/admin.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/routes/admin.py)
- Code: Whenever recruiters are approved, drives are verified, or user access states are toggled, the cache is wiped: `cache_clear_pattern("admin_stats:*")`

---

2. Celery Asynchronous Processing

Celery handles tasks that take longer than a standard HTTP request/response cycle, preventing server blockages or timeouts.

2.1. Connecting Celery to Flask Context

Celery tasks run in a standalone worker process separate from the Flask web server. To access the SQLite database,

Celery tasks must run within Flask's application context.

- File: [`backend/app.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/app.py)
- Code:

```
class ContextTask(celery_app.Task):
    def __call__(self, *args, **kwargs):
        with app.app_context():
            return self.run(*args, **kwargs)
celery_app.Task = ContextTask
```

- How it works: By subclassing Celery's `Task` class, every background task execution automatically wraps inside `with app.app_context():`, letting SQLAlchemy queries execute inside the worker.

2.2. Asynchronous Request Lifecycle (CSV Export)

When a student wants to export their application history to a CSV file:

1. Frontend Dispatches Request: The Vue client triggers a `POST` request to `/api/students/export-applications`.
2. Task Queued in Redis: The backend route [`backend/routes/students.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/routes/students.py) imports the task and dispatches it:

```
task = export_student_applications_task.delay(profile.id)
```

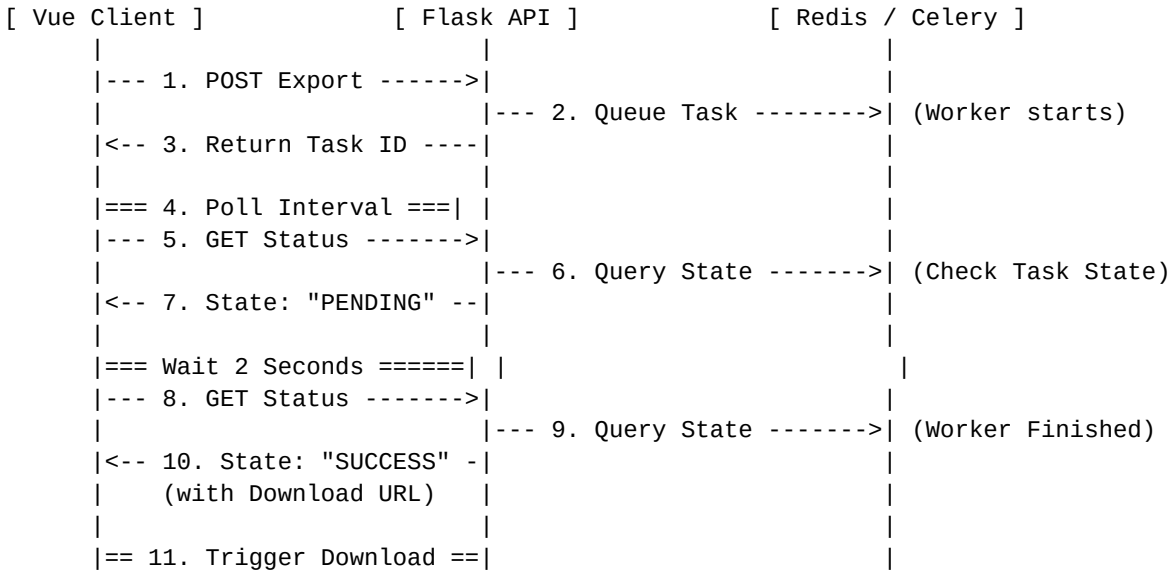
The `.delay()` method serializes the parameters and sends a task message to the Redis queue.

3. Immediate 202 Accepted Response: The Flask server immediately returns a `202 Accepted` status with the task's unique ID (`task.id`). The HTTP request terminates in milliseconds, keeping the client interface responsive.
4. Worker Execution: The Celery background worker fetches the task from Redis, runs `export_student_applications_task` in [`backend/tasks/csv_export.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/tasks/csv_export.py), generates the CSV file, and writes the filename as the task's result in the Redis backend.

---

3. Polling Mechanics

Because HTTP is a stateless request/response protocol, the server cannot push a file to the browser once a background Celery task completes. The client must ask the server for the status at regular intervals -- this is called polling.



3.1. The Polling Endpoint

- File: [`backend/routes/students.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/routes/students.py)

The frontend uses the following endpoint to check task state:

```
@students_bp.route("/export-status/<string:task_id>", methods=["GET"])
```

```
def export_status(task_id):
    from celery.result import AsyncResult
    res = AsyncResult(task_id, app=celery_app)

    if res.state == "SUCCESS":
        return jsonify({
            "status": "success",
            "download_url": f"/uploads/exports/{res.result}"
        }), 200
    elif res.state == "FAILURE":
        return jsonify({
            "status": "failure",
            "message": str(res.info)
        }), 500
    else:
        # State is PENDING, STARTED, or RETRY
        return jsonify({"status": "pending"}), 200
```

3.2. Client-side Polling Loop

- File: [`frontend/js/components/StudentDashboard.js`](file:///c:/Users/bachc/Downloads/projectiitm/frontend/js/components/StudentDashboard.js)

The Vue component manages the polling lifecycle:

1. Start Polling: Once the `task_id` is received, `pollExportStatus(taskId)` is called.
2. Interval Loop: The function initializes a `setInterval` loop to query the endpoint every 2 seconds:

```
    pollExportStatus(taskId) {
        const interval = setInterval(async () => {
            const res = await fetch(`/api/students/export-status/${taskId}`);
            const data = await res.json();

            if (data.status === 'success') {
                clearInterval(interval); // Stop the loop
                this.isExporting = false;
                window.location.href = data.download_url; // Download file
            } else if (data.status === 'failure') {
                clearInterval(interval); // Stop the loop
                this.isExporting = false;
                alert("CSV Export failed: " + data.message);
            }
        }, 2000);
    }
```

3. Completion: When the response state is `success`, `clearInterval` terminates the loop, and the browser is redirected to the file download URL to fetch the compiled CSV.

---

4. Scheduled Beat Tasks (Batch Jobs)

Celery Beat acts as a scheduler. It pushes tasks into the Redis queue at specified intervals based on a cron-like schedule.

4.1. Task Schedule Definitions

- File: [`backend/app.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/app.py)

```
beat_schedule={
    "send-daily-reminders": {
        "task": "tasks.daily_reminders.send_daily_reminders",
        "schedule": crontab(hour=0, minute=0)
    },
```

```
    "generate-monthly-report": {
        "task": "tasks.monthly_report.generate_monthly_report",
        "schedule": crontab(day_of_month=1, hour=0, minute=0)
    }
}
```

4.2. Daily Reminders Task

- File: [`backend/tasks/daily_reminders.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/tasks/daily_reminders.py)
- Execution Flow:

1. Calculates dates between `today` and `today + 3 days`.
2. Queries all approved jobs (`Job.query.filter_by(status="approved")`).
3. Filters drives whose application deadlines fall within this 3-day window.
4. Loops through all students and queries if they have already applied (`Application.query.filter_by(student_id=s.id, job_id=job.id).first()`).
5. For any student who has not applied, it appends a warning message to the notification log file `backend/uploads/notifications.log`.

4.3. Monthly Activity Reports

- File: [`backend/tasks/monthly_report.py`](file:///c:/Users/bachc/Downloads/projectiitm/backend/tasks/monthly_report.py)
- Execution Flow:

1. Runs on the first day of the month.
2. Calculates the boundary timestamps for the previous calendar month.
3. Queries the database to count:
   - Drives conducted during that month.
   - Applications submitted during that month.
   - Successful student selections during that month.
4. Queries the total registered student count and approved recruiter count.
5. Injects these metrics into a styled HTML template.
6. Writes the report to `backend/uploads/reports/monthly_report_[year]_[month].html` and logs a success alert in `notifications.log`.